PROJETO DE EXTENSÃO UNIVERSITÁRIA • HORTA COMUNITÁRIA

SISTEMA DE IRRIGAÇÃO AUTOMATIZADA COM ESP32

Guia Didático do Firmware, Arquitetura, Pinagem e Funcionamento Passo a Passo

---

Para quem é este material?
Este guia foi elaborado sob medida para estudantes, voluntários, agricultores urbanos e entusiastas que **não possuem familiaridade prévia com programação de microcontroladores** ou termos complexos de engenharia de software.

Aqui, cada linha de código é explicada através de analogias simples do dia a dia, e cada pino do ESP32 é apresentado com diagramas visuais e explicações práticas de onde ligar cada componente.

Hardware Principal:	ESP32 DevKit V1 (Dual Core 240MHz, 30 pinos)
Modo de Operação:	100% Offline (Rede Wi-Fi Própria / SoftAP sem necessidade de internet)
Linguagem e Ferramentas:	C++ moderno com Arduino Framework sob PlatformIO
Mecanismos de Segurança:	Cooldown (10s), Orçamento Diário (15m), Teto Máximo (5m), Parada de Emergência
Status da Montagem:	Fase de Mock/Testes de Bancada (LED Onboard simulando a Válvula)

Versão 1.0 • Documentação Técnica e Educacional Completa

1. O Que É Este Projeto e Por Que Ele Existe?

Imagine ter uma horta comunitária em um espaço aberto, praça ou escola onde as plantas precisam de água todos os dias nos horários mais frescos (de manhã cedo e no final da tarde). Nem sempre há alguém disponível para ligar a mangueira, e muitas vezes esse local **não tem internet Wi-Fi** nem um computador por perto. Como resolver isso de forma barata, sustentável e confiável?

A resposta é este projeto! Ele utiliza um chip minúsculo chamado **ESP32** para assumir o controle completo da irrigação. O projeto foi estruturado com uma premissa fundamental: **ele é 100% autossuficiente e offline**.

■ DICA TÉCNICA: Como o sistema funciona sem internet?

O ESP32 não precisa do roteador da sua casa. Ele mesmo gera a sua própria rede Wi-Fi privativa (com o nome **'Irrigacao-Comunitaria'**). Qualquer pessoa com um smartphone pode se aproximar da caixa do sistema, conectar nesse Wi-Fi e abrir o painel de controle pelo navegador de internet digitando 192.168.4.1. Funciona até mesmo no meio de uma fazenda sem sinal de celular!

Divisão de Responsabilidades no Projeto

Este projeto universitário foi dividido entre duas frentes de trabalho coordenadas:

Equipe de Software (Este Código)	Equipe de Montagem Física / Hidráulica
• Criar o cérebro lógico do sistema no ESP32. • Gerar as páginas de controle que aparecem no celular. • Garantir travas de segurança para nunca faltar água e nem inundar a horta. • Guardar o histórico permanente de tudo o que aconteceu.	• Instalar os canos de água, mangueiras gotejadoras e caixa d'água. • Conectar a válvula solenoide (a 'torneira elétrica') à rede hidráulica. • Fazer a ligação física dos fios na placa do ESP32. • Acondicionar o circuito em uma caixa estanque contra chuva e poeira.

■ ANALOGIA DO COTIDIANO: Autonomia e Resiliência

Por rodar 100% no ESP32, se faltar luz na horta, o sistema não desconfigura. Ao retornar a energia, o chip religa automaticamente, reconstrói o Wi-Fi e retoma o monitoramento da horta sem intervenção humana.

2. Dicionário Descomplicado: Decodificando as Nomenclaturas

A programação de microcontroladores é repleta de termos técnicos em inglês. A tabela abaixo traduz cada conceito para termos claros e analogias práticas do nosso dia a dia:

Termo Técnico	Nome Amigável	O que significa na prática?
ESP32	O Cérebro Eletrônico	Placa compacta com processador veloz, memória e antena de rádio (Wi-Fi e Bluetooth) embutidos em um único chip. Custa barato e consome pouquíssima energia.
Firmware	O Software de Fábrica	Diferente de um programa de computador comum instalado no Windows, o firmware é gravado na 'alma' do chip. Ao ligar na tomada, ele já acorda rodando esse código.
GPIO (General Purpose I/O)	Pinos de Entrada e Saída	Perninhas metálicas da placa. Servem como 'dedos' para ligar coisas (Saída: manda energia para o relé abrir a água) ou como 'olhos' para ler dados (Entrada: mede a umidade).
HIGH e LOW	Ligado e Desligado	Na eletrônica digital, tudo é 0 ou 1. **HIGH** significa colocar energia no pino (3.3V, ativando a válvula). **LOW** significa desligar a energia (0V, fechando a torneira).
SoftAP (Ponto de Acesso)	Roteador Próprio	Em vez de conectar em uma rede externa, o próprio ESP32 se transforma em um roteador sem fio, permitindo que o celular se conecte diretamente nele.
SPIFFS / Flash	O Pen Drive Interno	Área de memória permanente dentro do ESP32. Se faltar luz ou o sistema reiniciar, as configurações salvas e o histórico de regas continuam intactos.
Mock (Simulação)	O Dublê de Testes	Enquanto a válvula hidráulica não chega, o código acende o LED azul da própria placa (GPIO 2) para provar visualmente que o acionamento funciona.
RTC (Real Time Clock)	O Relógio de Bateria	Pequeno módulo acessório com bateria tipo moeda (DS3231). Mantém a hora e os minutos correndo certinho mesmo com o ESP32 desligado da tomada.
millis() vs. delay()	Cronômetro x Cochilo	delay(1000) faz o processador dormir por 1 segundo travando tudo (ele para de responder o Wi-Fi). Já millis() funciona como um cronômetro: o processador checa o tempo sem parar de atender o usuário.
PlatformIO	O Mecânico de Build	Ferramenta moderna de desenvolvimento que substitui a antiga Arduino IDE. Baixa as bibliotecas sozinho e organiza a compilação com estabilidade profissional.

3. Diagrama de Pinagem do ESP32 DevKit V1

O ESP32 DevKit V1 de 30 pinos concentra toda a conectividade do projeto. A figura abaixo identifica cada terminal, destacando em cores vivas os pinos utilizados no firmware e as portas reservadas para expansão:



Figura 1: *Diagrama de pinagem do ESP32 DevKit V1 (30 pinos) com identificação das funções do projeto.*

Resumo Rápido dos Pinos Utilizados:

Pino	Cor	Função	Descrição
GPIO 2 (D2)	Azul	**Válvula / LED Onboard**	Ativa o relé que abre a válvula (atualmente acende o LED azul).
GPIO 21 (SDA)	Roxo	**Dados I2C (RTC)**	Linha de comunicação de dados com o relógio de precisão DS3231.
GPIO 22 (SCL)	Roxo	**Clock I2C (RTC)**	Linha de sincronismo de relógio com o módulo DS3231.
GPIO 36 (VP)	Verde Petróleo	**Sensor Umidade (ADC)**	Entrada analógica reservada para leitura capacitiva do solo.
VIN (5V) e GND	Vermelho/Preto	**Alimentação Principal**	Entrada de 5V da fonte e retorno negativo comum de corrente.

Esquema Elétrico Didático do Sistema Completo

Abaixo está o esquema elétrico de fácil compreensão que a equipe de montagem física utilizará para interligar a placa aos componentes reais na caixa de controle:

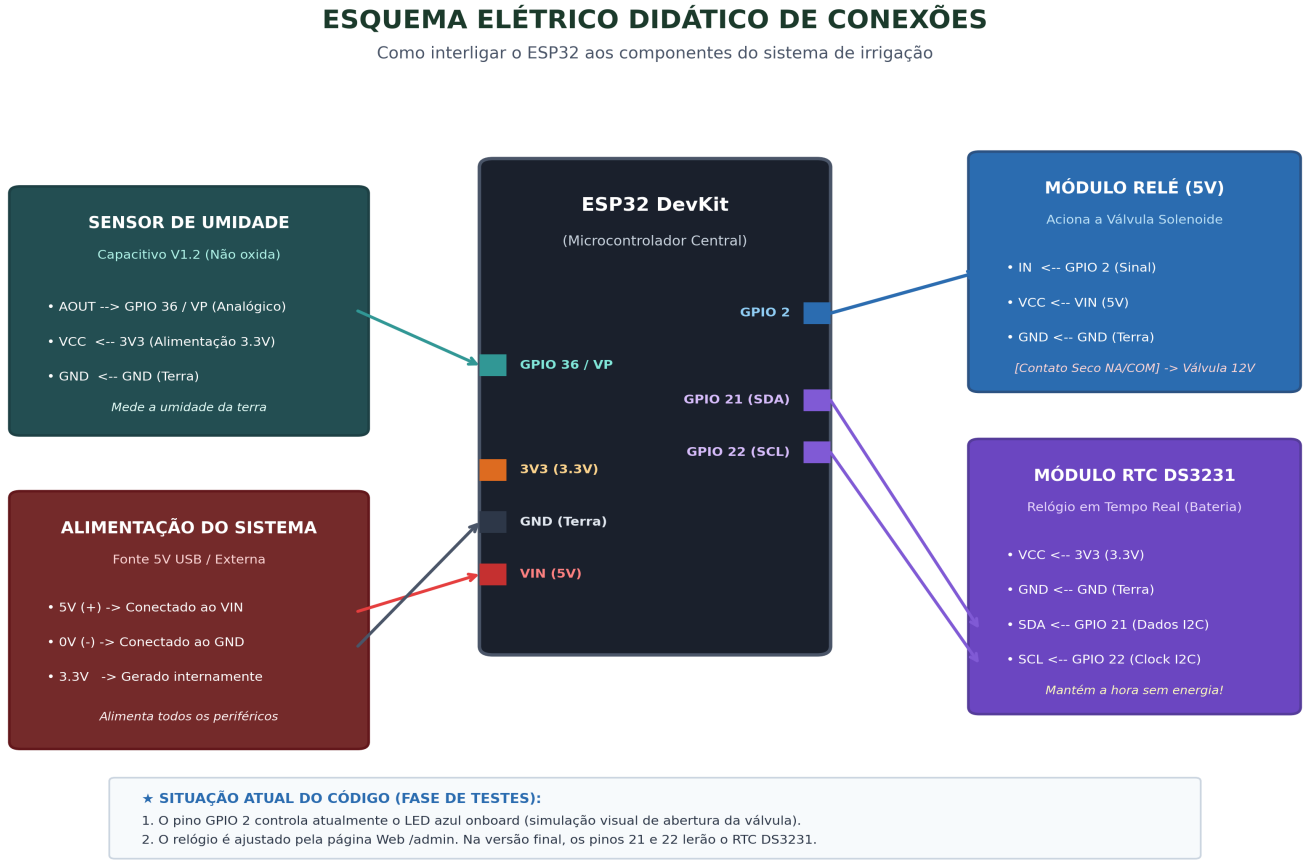


Figura 2: *Esquema de interligação dos módulos (Relé 5V, Válvula 12V, RTC DS3231, Sensor e Fonte).*

■ DICA TÉCNICA: Por que precisamos de um Módulo Relé para acionar a válvula?

O ESP32 é um processador que trabalha com níveis lógicos delicados: seus pinos fornecem apenas 3.3 Volts e corrente baixíssima (cerca de 20 a 40 miliamperes). As válvulas solenoide de água exigem 12 Volts e correntes elevadas para puxar o êmbolo hidráulico. Ligar a válvula diretamente no pino do ESP32 destruiria o chip na hora! O **Módulo Relé** atua como uma ponte segura: o ESP32 envia uma ordem de baixa voltagem e o relé chaveia a linha de alta voltagem com isolamento galvânico total.

Regra de Ouro da Eletrônica: O Terra Comum (GND)

Mesmo que a válvula use uma fonte de 12V e o ESP32 seja alimentado em 5V, o terminal **GND** de todas as placas e sensores deve ser conectado junto. O GND é o ponto de referência zero para que os sinais elétricos sejam interpretados corretamente por todos os chips.

4. Arquitetura Geral do Sistema (Visão Sistêmica)

Um software bem estruturado é organizado em blocos funcionais independentes. A imagem a seguir sintetiza a rota da informação, desde o celular do hortelão até o acionamento físico da água:

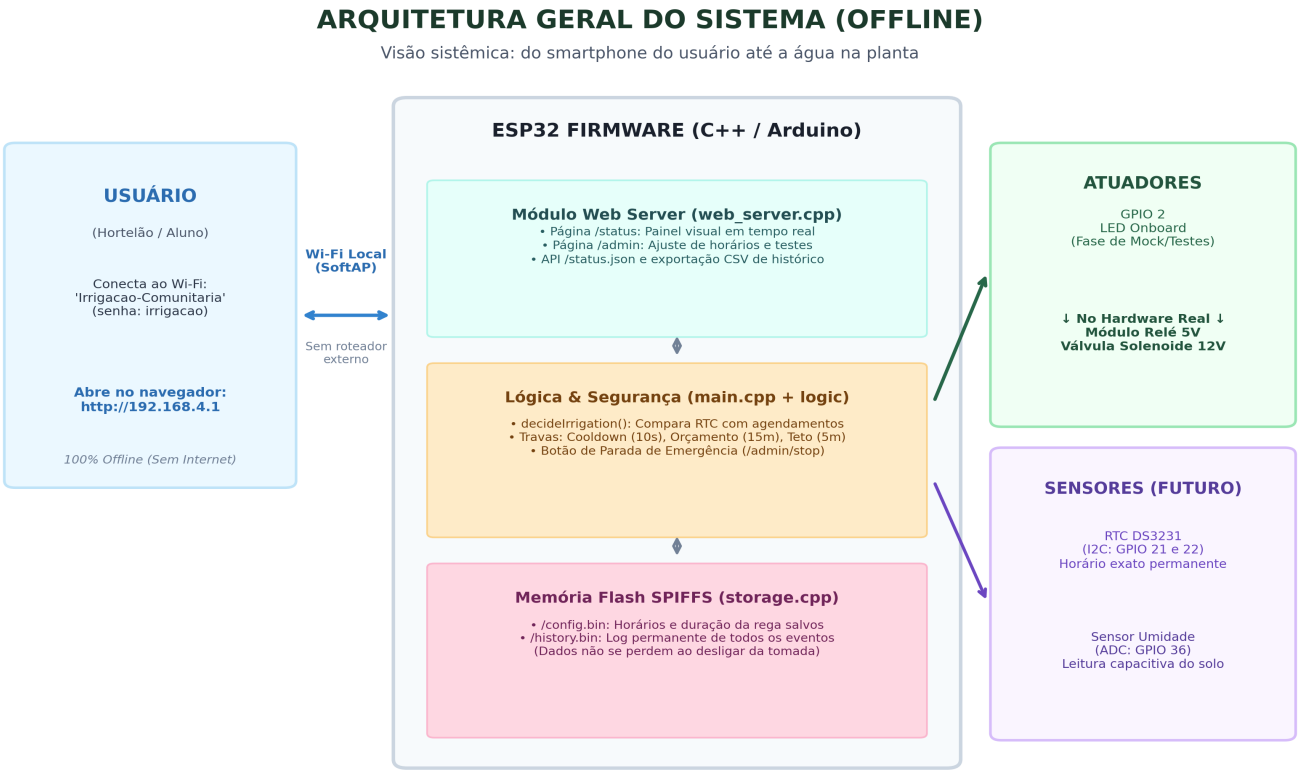


Figura 3: *Arquitetura em camadas do sistema offline (Usuário -> Firmware ESP32 -> Periféricos).*

Entendendo as Três Camadas do Sistema:

1. Camada do Usuário (Interface Humana): Qualquer smartphone comum se conecta à rede Wi-Fi local Irrigacao-Comunitaria (senha: irrigacao) e abre 192.168.4.1 no navegador. Não precisa instalar nenhum aplicativo na Play Store ou Apple Store!

2. Camada do Firmware (Cérebro Interno): Dividida em três partes:
• *Servidor Web Assíncrono (web_server.cpp):* Atende as requisições HTTP sem congelar a placa.
• *Lógica & Segurança (main.cpp e logic):* Compara os horários e aplica as travas de proteção.
• *Memória Flash SPIFFS (storage.cpp):* Salva a programação e o histórico de forma permanente.

3. Camada Física (Atuadores e Sensores): O pino GPIO 2 comanda o relé da válvula de água. No futuro, os pinos I2C lerão o RTC DS3231 e o pino ADC 36 medirá a umidade do solo.

5. Raio-X dos Códigos-Fonte (Parte 1: Estrutura Geral)

A pasta do projeto contém códigos modulares escritos em C++ moderno para a plataforma Arduino/ESP32. Vejamos o inventário completo dos arquivos e a responsabilidade de cada um:

Arquivo	Papel no Projeto	O que ele faz em detalhes?
platformio.ini	Configuração de Build	Diz ao compilador qual placa usar (esp32dev), a velocidade serial (115200) e baixa as bibliotecas externas necessárias.
include/Config.h	Contrato de Parâmetros	Define a estrutura de dados das programações (ScheduleSlot) e as constantes fixas de segurança (cooldown, cota diária).
include/History.h	Contrato de Histórico	Define a estrutura HistoryEntry e o enumerador TriggerReason (motivos de acionamento ou bloqueio de rega).
include/Routes.h	Contrato de URLs Web	Centraliza todos os caminhos do servidor HTTP (/status, /admin, /admin/config, etc.) para evitar erros de digitação.
include/Storage.h	Contrato de Persistência	Declara as assinaturas das funções de gravação e leitura na memória flash (loadConfig, saveConfig, etc.).
src/irrigation_logic.*	Cérebro Matemático Puro	Função sem hardware: compara a hora atual com os horários cadastrados e diz se deve irrigar.
src/storage.cpp	Arquivista da Flash	Implementa a gravação e leitura direta em binário bruto nos arquivos /config.bin e /history.bin no SPIFFS.
src/web_server.*	Garçom e Vitrine Web	Monta as páginas HTML/CSS inline 100% offline, valida entradas de formulários e exporta o histórico em CSV.
src/main.cpp	Maestro Operacional	Inicializa o ESP32 (setup), sobe o Wi-Fi SoftAP, sincroniza o relógio e executa o laço contínuo de vigilância (loop).

Detalhamento: platformio.ini e Config.h

Arquivo: platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    ESP32Async/ESPAsyncWebServer
    ESP32Async/AsyncTCP
    adafruit/RTClib
```

O que esse trecho faz: Configura o ambiente para compilação no ESP32 clássico, ajusta a porta serial e baixa as bibliotecas de servidor web assíncrono e relógio RTC automaticamente.

Arquivo: include/Config.h (Trecho de Segurança)

```
constexpr uint16_t MAX_IRRIGATION_DURATION_SEC = 300; // teto máximo de 5 minutos
constexpr uint16_t IRRIGATION_COOLDOWN_SEC = 10;      // pausa de 10s entre acionamentos
constexpr uint32_t DAILY_WATER_BUDGET_SEC = 900;      // limite de 15 min de água por dia
```

O que esse trecho faz: Constantes fixadas diretamente no firmware. Não podem ser alteradas por acidente pela interface web, servindo de barreira de proteção definitiva.

5. Raio-X dos Códigos (Parte 2: Lógica Pura e Persistência)

A) src/irrigation_logic.cpp — A Lógica Desacoplada de Hardware

Um princípio clássico de engenharia de software é: *mantenha a lógica matemática pura separada dos pinos físicos*. A função decideIrrigation() não mexe em fios nem em relés; ela apenas compara números:

Arquivo: src/irrigation_logic.cpp

```
IrrigationDecision decideIrrigation(const IrrigationConfig& config,
                                    uint8_t currentHour,
                                    uint8_t currentMinute) {
    for (uint8_t i = 0; i < config.scheduleCount; i++) {
        const ScheduleSlot& slot = config.schedules[i];
        if (slot.enabled && slot.hour == currentHour && slot.minute == currentMinute) {
            return {true, config.irrigationDurationSec};
        }
    }
    return {false, 0};
}
```

O que esse trecho faz: A função percorre os horários agendados. Se um slot estiver ativo e a hora e minuto baterem com o relógio, ela devolve 'true' e o tempo em segundos que a água deve correr.

B) src/storage.cpp — Persistência Binária Eficiente em Flash (SPIFFS)

Em computadores comuns, é comum salvar arquivos em JSON ou bancos de dados SQL. Porém, no ESP32, essas bibliotecas gastam muita memória RAM preciosa. O projeto utiliza uma estratégia genial: **salvar estruturas C++ em bytes crus (binário)**:

Arquivo: src/storage.cpp (Gravação Direta de Structs)

```
bool saveConfig(const IrrigationConfig& config) {
    File file = SPIFFS.open(CONFIG_PATH, FILE_WRITE);
    size_t written = file.write(reinterpret_cast<const uint8_t*>(&config;),
                                sizeof(IrrigationConfig));
    file.close();
    return written == sizeof(IrrigationConfig);
}
```

O que esse trecho faz: A função abre o arquivo '/config.bin' e grava o bloco de memória exato da estrutura IrrigationConfig de uma só vez. É ultra-rápido (frações de milissegundo) e consome zero de RAM extra.

■ DICA TÉCNICA: O que acontece no primeiro boot do ESP32?

No primeiro boot da placa recém-saída da caixa, o arquivo /config.bin ainda não existe. A função loadConfig() percebe isso automaticamente e preenche a configuração com valores seguros padrão (zero horários ativos e duração de 10 segundos para testes). Em seguida, ela formata o sistema de arquivos com SPIFFS.begin(true) sem apresentar erros.

5. Raio-X dos Códigos (Parte 3: Servidor Web e Loop Central)

A) src/web_server.cpp — Servidor Web Responsivo e 100% Offline

O servidor HTTP embutido atende tanto as páginas visuais em HTML para celulares quanto APIs em formato JSON. Todo o estilo CSS utiliza variáveis modernas e flexbox, renderizando perfeitamente no celular sem depender de CDN:

Arquivo: src/web_server.cpp (Exportação de Histórico em CSV)

```
server.on(AsyncURIMatcher::exact(Routes::ADMIN_HISTORY), HTTP_GET, [](AsyncWebServerRequest* request) {
    AsyncWebServerResponse* response = request->beginResponse(200, "text/csv", buildHistoryCsv());
    response->addHeader("Content-Disposition", "attachment; filename=history.csv");
    request->send(response);
});
```

O que esse trecho faz: Ao clicar em 'Baixar CSV', o ESP32 lê o arquivo /history.bin, traduz os timestamps para datas legíveis (ex: 24/09/2026 17:30) e envia como download direto para o navegador.

B) src/main.cpp — O Maestro do Sistema (Setup e Loop)

O arquivo main.cpp conecta todas as peças. Ele é responsável pelo ciclo de vida da placa:

Arquivo: src/main.cpp (Laço Principal Não-Bloqueante)

```
void setup() {
    pinMode(VALVE_PIN, OUTPUT);        // Configura pino GPIO 2
    loadConfig(config);                 // Carrega agendamentos da memória
    WiFi.softAP(AP_SSID, AP_PASSWORD); // Cria o Wi-Fi 'Irrigacao-Comunitaria'
    server.begin();                     // Coloca o servidor web no ar
}

void loop() {
    updateValve();         // Fecha a válvula se o tempo estipulado tiver passado
    evaluateIrrigation();  // Avalia a cada 1 segundo se bateu o horário agendado
}
```

O que esse trecho faz: Perceba a ausência de delay()! A função updateValve() compara o cronômetro millis() com o tempo limite da rega, mantendo o processador 100% livre para atender os usuários no Wi-Fi.

■ ANALOGIA DO COTIDIANO: Deduplicação de Minuto: Evitando Disparos Repetidos

Como a função evaluateIrrigation() roda a cada segundo e a irrigação é programada por hora e minuto (ex: 06:30), o minuto 30 dura 60 segundos inteiros! Sem uma proteção, o sistema tentaria religar a válvula 60 vezes seguidas. O código resolve isso com uma 'chave de minuto' (currentMinuteKey = hour * 60 + minute), garantindo que o disparo ocorra exatamente uma única vez por agendamento.

6. Rotinas de Segurança: A Blindagem do Sistema

Em hortas comunitárias, uma válvula travada aberta por falha de código pode esvaziar a caixa d'água ou afogar as plantações. O firmware foi construído com **quatro travas de segurança em cascata** que impedem qualquer desperdício:

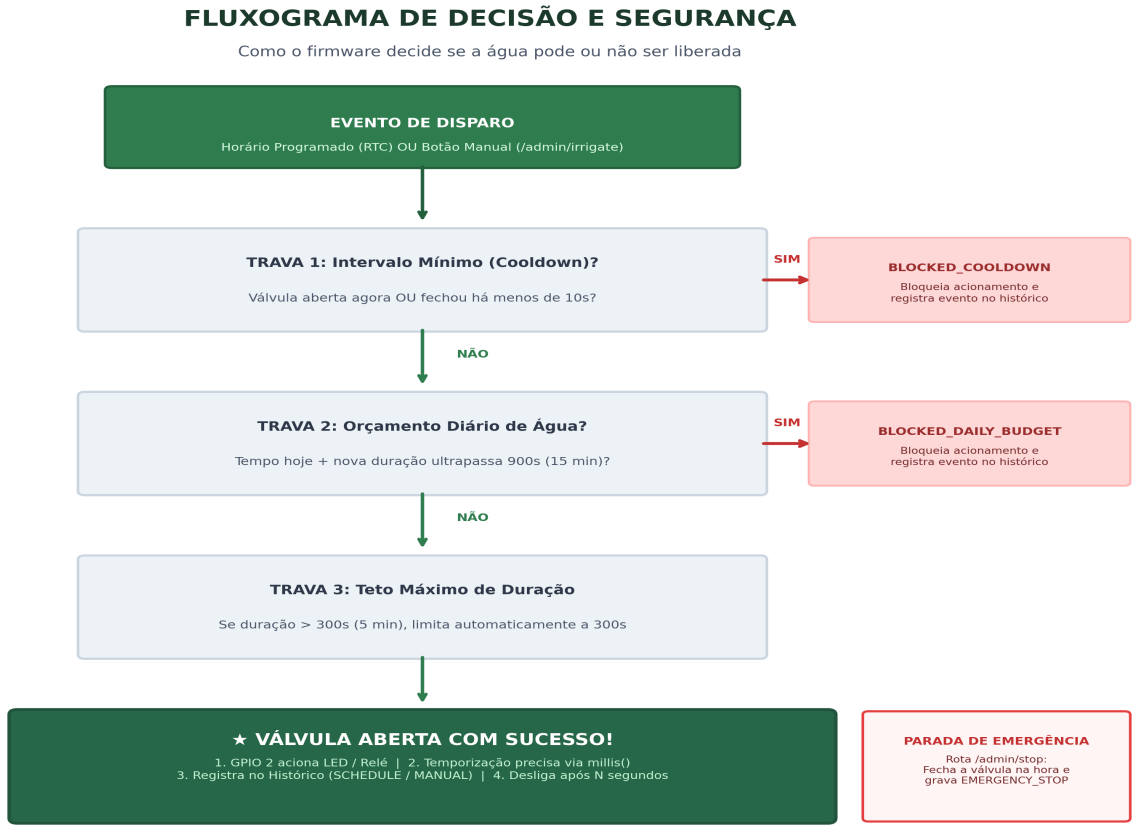


Figura 4: Fluxograma das quatro etapas de validação de segurança antes da liberação de água.

As Quatro Travas de Segurança Explicadas:

• **Trava 1: Intervalo Mínimo (Cooldown de 10s):** Impede acionamentos em rajada por cliques repetidos no botão web. Se a válvula acabou de fechar, aguarda no mínimo 10 segundos antes de reabrir, evitando o golpe de aríete nos canos.

• **Trava 2: Orçamento Diário de Água (900s / 15 min):** Soma o tempo total de torneira aberta durante o dia. Ao atingir 15 minutos acumulados, bloqueia qualquer novo acionamento até o dia seguinte. Proteção vital contra encharcamento!

• **Trava 3: Teto Absoluto por Acionamento (300s / 5 min):** Nenhum acionamento manual ou agendado ultrapassa 5 minutos. Se o usuário digitar 9999 segundos na interface, a função openValve() poda o valor automaticamente para 300 segundos.

• **Trava 4: Parada de Emergência (/admin/stop):** Botão vermelho de ação instantânea. Ao ser clicado, desliga a válvula no mesmo milissegundo e grava um registro com o motivo EMERGENCY_STOP no histórico.

■■ ATENÇÃO / SEGURANÇA: Auditoria Completa de Bloqueios no Histórico

Quando uma tentativa de irrigar é barrada pelas travas, o sistema grava a recusa no arquivo de histórico com o motivo exato: BLOCKED_COOLDOWN ou BLOCKED_DAILY_BUDGET. Nada ocorre de forma invisível!

7. Como Usar na Prática: Guia do Hortelão

A operação foi simplificada ao máximo para que qualquer pessoa consiga utilizar o sistema em campo através da câmera do celular, sem precisar digitar comandos ou instalar aplicativos:

PASSO 1: Conectar ao Wi-Fi Aponte a câmera para conectar: **SSID:** Irrigacao-Comunitaria **Senha:** irrigacao	**PASSO 2: Abrir o Painel** Aponte a câmera para abrir o painel: **Endereço direto:** http://192.168.4.1/status

Figura 5: *QR Codes para fixação na tampa da caixa hermética da horta comunitária.*

Navegação pelas Telas do Sistema:

1. Tela de Status Geral (/status):
Painel público e intuitivo. Apresenta uma luz em destaque (badge): verde (**'Irrigando agora'**) ou cinza (**'Aguardando'**). Exibe o relógio do sistema, quando foi a última rega e a lista de horários ativos. A página se atualiza sozinha a cada 5 segundos via JavaScript sem precisar recarregar o navegador.

2. Tela Administrativa (/admin):
Painel de controle para os responsáveis pela horta. Permite:
• *Ajustar Horário do Sistema:* Como ainda não há o RTC físico, use o seletor de data/hora do celular para sincronizar o relógio.
• *Programar Agendamentos:* Habilita os slots de irrigação (hora e minuto) e a duração desejada.
• *Irrigação Manual de Teste:* Abre a torneira sob demanda para verificar a vazão dos canos.
• *Parada de Emergência:* Fecha a válvula imediatamente em caso de vazamento.
• *Baixar Histórico em CSV:* Exporta o relatório completo de eventos para abrir no Microsoft Excel.

■■ ATENÇÃO / SEGURANÇA: Atenção para a Fase de Entrega à Comunidade:

Nesta fase de desenvolvimento em laboratório, a exigência de senha de acesso foi temporariamente desligada para facilitar os testes de bancada. Antes de colocar o equipamento na horta, reative a senha nas rotas /admin para que pessoas não autorizadas não alterem as programações.

8. Do Mock ao Hardware Real: Roteiro de Montagem

O firmware já está pronto e testado em bancada (com o LED azul onboard simulando a válvula). Quando a equipe de montagem física receber as peças definitivas, basta seguir este roteiro de instalação:

• **Passo 1: Conexão do Módulo Relé de 5V:** Ligue o pino **IN** do relé no pino **GPIO 2** do ESP32, o **VCC** no pino **VIN (5V)** e o **GND** no **GND**. O código já aciona esse pino sem necessidade de alterar nenhuma linha de programação!

• **Passo 2: Ligação da Válvula Solenoide de 12V:** A válvula possui dois polos. Interrompa o polo positivo passando pelo contato Normalmente Aberto (NA/COM) do relé e alimente pela fonte de 12V externa. Ao fechar o relé, a água corre.

• **Passo 3: Instalação do Relógio RTC DS3231:** Conecte o **VCC** em **3V3**, **GND** em **GND**, **SDA** no pino **GPIO 21** e **SCL** no pino **GPIO 22**. Descomente a rotina do RTC no main.cpp. A placa nunca mais perderá a hora ao ser desligada!

• **Passo 4: Integração do Sensor de Umidade Capacitivo:** Enterre o sensor V1.2 próximo à raiz das plantas. Ligue seu **VCC** em **3V3**, **GND** em **GND** e o terminal **AOUT** no pino **GPIO 36 (VP)**.

• **Passo 5: Acondicionamento Hermético (Caixa IP65):** O circuito deve ser fixado dentro de uma caixa plástica vedada com proteção IP65 (contra chuva, poeira e formigas). Os cabos devem passar por prensa-cabos emborrachados na parte inferior.

Checklist de Validação Física Antes de Ligar a Água:

Item a Verificar	Critério de Aceite	Status
Terra Comum (GND)	GND da fonte 12V, do ESP32 e dos módulos conectados juntos.	**[ ] OK**
Tensão no ESP32	Medir 5V estáveis no pino VIN e 3.3V no pino 3V3 com multímetro.	**[ ] OK**
Sentido da Válvula	A seta de fluxo no corpo da solenoide aponta para a horta.	**[ ] OK**
Estanqueidade	Abrir o registro de água com relé desligado: vazamento zero.	**[ ] OK**
Acionamento Manual	Clicar em 'Irrigar agora' via celular: relé atraca e água sai.	**[ ] OK**

9. Resolução de Dúvidas e Ficha Técnica de Bolso

P: O ESP32 perdeu as horas depois que faltou luz. O que aconteceu?

R: Nesta fase de bancada, o relógio externo de bateria (RTC) ainda não foi conectado. O horário fica guardado na memória RAM. Ao ligar, basta conectar pelo celular em /admin e salvar o horário atual.

P: Estou longe da horta e não encontro o Wi-Fi. Qual o alcance do sinal?

R: A antena compacta do ESP32 alcança de 15 a 30 metros em linha reta. É necessário se aproximar da caixa da horta para estabelecer conexão com o painel.

P: Apareceu 'Falha detectada' em vermelho na tela de status. O que significa?

R: Indica que a gravação na memória SPIFFS falhou (memória corrompida). Basta regravar a partição do sistema de arquivos via pio run -t uploadfs.

Ficha Técnica de Bolso (Guarde com o Equipamento)

Rede Wi-Fi (SSID):	Irrigacao-Comunitaria
Senha do Wi-Fi:	irrigacao
Endereço no Navegador:	http://192.168.4.1/status (Público) | http://192.168.4.1/admin (Config)
Pino da Válvula (Sinal):	GPIO 2 (Ativo em nível HIGH / 3.3V)
Pinos do Relógio RTC DS3231:	GPIO 21 (SDA) e GPIO 22 (SCL) via I2C
Pino do Sensor de Umidade:	GPIO 36 / VP (Leitura Analógica de 0 a 4095)
Limites de Segurança:	Cooldown: 10s | Orçamento Diário: 15 min (900s) | Teto Máximo: 5 min (300s)
Comandos de Gravação (PlatformIO):	pio run (Compilar) | pio run -t upload (Gravar) | pio run -t uploadfs (Limpar Flash)

Documentação técnica e didática desenvolvida para o Projeto de Extensão Universitária — Horta Comunitária Inteligente. Código-fonte sob licença livre e colaborativa. Todos os direitos reservados à equipe do projeto.